



SAPIENZA
UNIVERSITÀ DI ROMA

“SAPIENZA” UNIVERSITY OF ROME

FACULTY OF INFORMATION ENGINEERING,
COMPUTER SCIENCE AND STATISTICS

DEPARTMENT OF COMPUTER, CONTROL
AND MANAGEMENT ENGINEERING

MASTER'S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

Elective in Robotics

Lecture notes

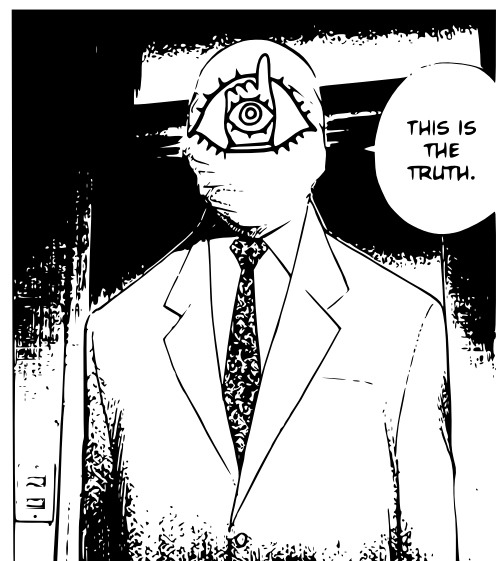
Author

Marco Casu

<https://github.com/CasuFrost>

September 23, 2026

This document summarizes and presents the topics for the Elective in Robotics course for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.



CONTENTS

I	Modeling and Control of Multi-Rotor UAVs	3
1	Introduction	4
1.1	Quadrotor Modeling	5
1.2	Quadrotor Control	9

Part I

**Modeling and Control of Multi-Rotor
UAVs**

CHAPTER

1

INTRODUCTION

UAV stands for **Unmanned Aerial Vehicle**, also known by other names depending on the domain, like *RPVs (Remotely Piloted Vehicles)* in the military domain, or *Drones* in common parlance. We will use all of the three names interchangeably. The methods presented are valid also for remotely operated or autonomous vehicles (at different levels of autonomy).

The term UAVs refers to a broad category of vehicles, in general, any aerial vehicle without a pilot on board is a UAV. We will examine UAVs composed of multiple rotors, in particular **quadrotors**.



Figure 1.1: Design-based classification.

The quadrotors are used in various situations:

- inspection, maintenance,
- communication networks,
- search and rescue operations,
- aerial transportation, manipulation.

These activities require a vertical, stationary and slow flight. A quadrotor is characterized by a high maneuverability, can remain suspended in the air without moving (hovering), and perform a *vertical take-off and landing (VTOL)*, unlike other vehicles like airplanes, that require a horizontal movement in order to take-off and land.

1.1 Quadrotor Modeling

We call VTOLs the vehicles that can perform a vertical take-off and landing, these are modeled with the North-East-Down (NED) reference frame:

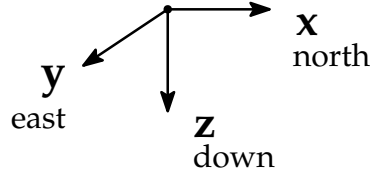


Figure 1.2: The NED reference frame.

This frame, often called SR_i , has the z axis pointing down to the center of the Earth. According to the dynamics of rigid bodies given by the Newton-Euler formalism, the translational dynamics of a body in the SR_i frame is given by

$$\sum_j \mathbf{F}_j = m\dot{\mathbf{V}} \quad (1.1)$$

where $\mathbf{F}_j$ are the external forces applied to the center of mass, and $\dot{\mathbf{V}}$ is the acceleration of the center of mass expressed in SR_i . The rotational dynamics is given by

$$\sum_j \mathbf{M}_j = J\dot{\boldsymbol{\Omega}} + \boldsymbol{\Omega} \times J\boldsymbol{\Omega} \quad (1.2)$$

where $\mathbf{M}_j$ are the external moments acting on the center of mass, $\boldsymbol{\Omega}$ is the angular velocity of the body expressed in the body frame, denoted SR_b and J is the inertia tensor, also expressed in SR_b .

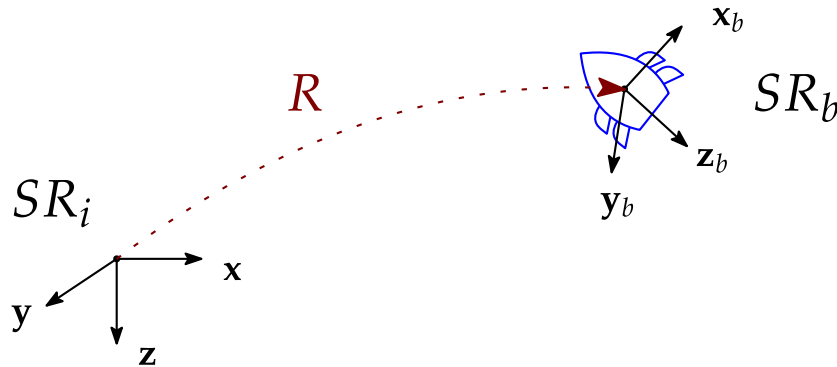


Figure 1.3: SR_b is the reference frame attached to the center of mass of the vehicle, R is the rotation matrix between the two frames.

A more explicit expression of the forces in Equation (1.1) is the following:

$$m\dot{\mathbf{V}} = -TR\mathbf{z} + \mathbf{F}_e(\mathbf{V}, \dot{\mathbf{V}}, \boldsymbol{\Omega}, \dot{\boldsymbol{\Omega}}, \mathbf{d}(t)) + R\Sigma_R\boldsymbol{\tau} \quad (1.3)$$

where:

- $\mathbf{d}(t)$ are the external disturbances that do not depend on the vehicle's state.
- $\mathbf{F}_e$ are the external forces, that depend on the state.

- T is the **thrust**, that depends on the number and allocation of the rotors, while $-\mathbf{z}$ is the direction pointing upward (according to the NED frame as in Figure 1.2). R is the rotation matrix that describes the orientation of our vehicle with respect to the fixed NED frame SR_i ,
- hence the product $-TR\mathbf{z}$ is the main upward thrust.
- $\boldsymbol{\tau}$ is the torque generated by the propellers.
- $R\Sigma_R\boldsymbol{\tau}$ are the parasitic forces generated by the rotors/actuators. The coupling matrix Σ_R maps the torques/rotations of the motors into small translational lateral forces (then rotated into SR_i via R).

We know that the derivative of the rotation matrix R depends on the skew-symmetric matrix of the angular velocity:

$$\dot{R} = R \cdot S(\boldsymbol{\Omega}) \quad (1.4)$$

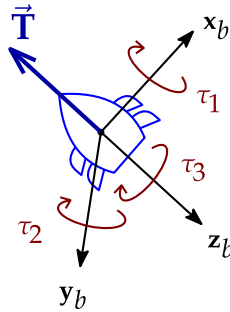
$S(\boldsymbol{\Omega})$ is important since it appears in the explicit expression of the term $J\dot{\boldsymbol{\Omega}}$ in Equation (1.2):

$$J\dot{\boldsymbol{\Omega}} = -S(\boldsymbol{\Omega})J\boldsymbol{\Omega} + \boldsymbol{\tau} + \boldsymbol{\tau}_e(\mathbf{V}, \dot{\mathbf{V}}, \boldsymbol{\Omega}, \dot{\boldsymbol{\Omega}}, \mathbf{d}(t)) + \boldsymbol{\tau}_g + \Sigma_T T \mathbf{z} \quad (1.5)$$

where:

- $\boldsymbol{\tau}_e$ are the torques induced by the external forces.
- $\Sigma_T T \mathbf{z}$ is a parasitic moment. Σ_T is a matrix that represents the geometric misalignment between the effective axis of the total thrust T and the center of mass of the vehicle. If the thrust vector does not pass exactly through the center of gravity, it generates a parasitic torque proportional to the thrust itself.
- $\boldsymbol{\tau}_g$ is the internal gyroscopic moment due to the inertia of the propellers rotating at high speed (particularly relevant in quadcopters when they change attitude rapidly).

We denote $\vec{T} = -T\mathbf{z}_b$ the push in the opposite direction of $\mathbf{z}_b$ expressed in the vehicle frame SR_b .



A quadrotor is a UAV with four motors located at the extremities of a cross-shaped frame (Figure 1.4) and is controlled by imposing a desired angular speed on each rotor.

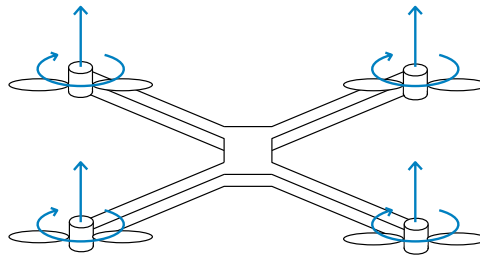


Figure 1.4: Schematic of a quadrotor.

Let's identify a disc on each rotor, that follows the profile of the propellers: we denote f_i the scalar force on each propeller, which is perpendicular to the disc, its value is:

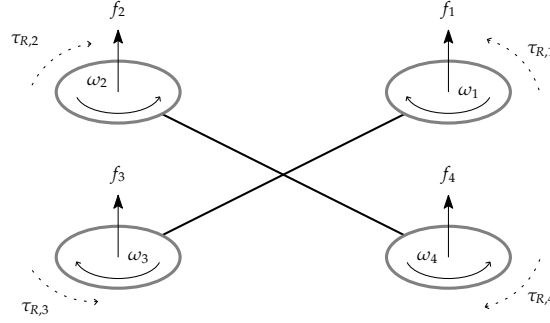
$$f_i = b\omega_i^2 \quad (1.6)$$

where ω_i is the angular speed of the i -th propeller, and b is a constant *thrust factor*, that depends on the rotor geometry and profile, and on air density. The rotational speed of each blade is controlled by a low level controller.

We denote $\tau_{R,i}$ the drag torque generated by the i -th rotor as it spins in the air:

$$\tau_{R,i} = d\omega_i^2 \quad (1.7)$$

where d is a *drag factor* that, as the thrust factor, depends on the rotor.



We denote φ, ϑ, ψ the roll-pitch-yaw angles of the vehicle with respect to the fixed reference frame SR_i , hence the already mentioned rotation matrix is:

$${}^iR_b = \begin{pmatrix} \cos \psi \cos \vartheta & \cos \psi \sin \vartheta \sin \varphi - \sin \psi \cos \varphi & \cos \psi \sin \vartheta \cos \varphi + \sin \psi \sin \varphi \\ \sin \psi \cos \vartheta & \sin \psi \sin \vartheta \sin \varphi + \cos \psi \cos \varphi & \sin \psi \sin \vartheta \cos \varphi - \cos \psi \sin \varphi \\ -\sin \vartheta & \cos \vartheta \sin \varphi & \cos \vartheta \cos \varphi \end{pmatrix}. \quad (1.8)$$

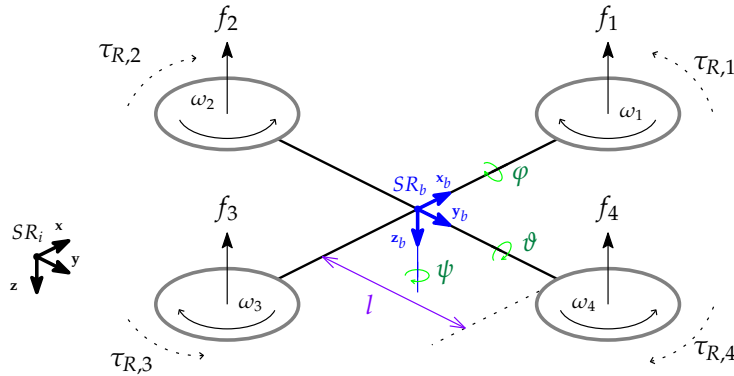


Figure 1.5: Configuration of the quadrotor.

The linear velocity $\mathbf{V}$ of our quadrotor is expressed in SR_i while the angular velocity $\mathbf{\Omega}$ is expressed in SR_b , we can make the reference system explicit using superscripts as follows:

$${}^i\mathbf{V} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}, \quad {}^b\mathbf{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix}. \quad (1.9)$$

Let $\omega_1, \omega_2, \omega_3, \omega_4$ be our control inputs (the velocity of the rotors), f_i and $\tau_{R,i}$ are derived from these quantities as in Equations (1.6) and (1.7). The total thrust will be:

$$T = f_1 + f_2 + f_3 + f_4. \quad (1.10)$$

We recall that on the quadrotor, a force $\vec{T} = -T\mathbf{z}_b$ acts in the same direction of $\mathbf{z}_b$ but with the opposite sense. Intuitively, if the forces f_i are identical, we can expect the quadrotor to accelerate towards $-\mathbf{z}_b$ without rotating on itself. The presence of these forces, if they are not identical, will cause a rotation of

the body. The torques generated on the axes $\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b$ are the following:

$$\tau_\varphi = l(f_2 - f_4), \quad (1.11)$$

$$\tau_\vartheta = l(f_1 - f_3), \quad (1.12)$$

$$\tau_\psi = -\tau_{R,1} + \tau_{R,2} - \tau_{R,3} + \tau_{R,4}. \quad (1.13)$$

l represents the arm length (or lever arm) of the quadrotor, as shown in Figure 1.5.

For simplicity, we can assume that the inertia matrix shown in Equation (1.2) is diagonal:

$$J = \begin{pmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{pmatrix} \quad (1.14)$$

the angular velocity instead can be expressed in function of the roll-pitch-yaw angles:

$$\boldsymbol{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} 1 & 0 & -\sin \vartheta \\ 0 & \cos \varphi & \cos \vartheta \sin \varphi \\ 0 & -\sin \varphi & \cos \vartheta \cos \varphi \end{pmatrix} \begin{pmatrix} \dot{\varphi} \\ \dot{\vartheta} \\ \dot{\psi} \end{pmatrix}. \quad (1.15)$$

Let's see in details what are the applied forces and moments. We explicit the sum of forces of the translational dynamics (that we recall is expressed in the fixed frame SR_i):

$$\sum_j \mathbf{F}_j = \underbrace{\begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix}}_{\text{weight}} + \underbrace{{}^i R_b \begin{pmatrix} 0 \\ 0 \\ -T \end{pmatrix}}_{\text{actuation}} + \mathbf{F}_A + \mathbf{F}_D \quad (1.16)$$

where $\mathbf{F}_A$ are the aerodynamics related forces, which mainly include the aerodynamic resistance of the body and arms of the aircraft to advancement, as well as any parasitic lift effects generated by the shape of the drone or by the rotor discs in translation, while $\mathbf{F}_D$ are the disturbances (for example, wind).

The sum of moments is the following:

$$\sum_j \mathbf{M}_j = \underbrace{\begin{pmatrix} L_\varphi \\ L_\vartheta \\ L_\psi \end{pmatrix}}_{\text{gyroscopic effects}} + \underbrace{\begin{pmatrix} \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix}}_{\text{actuation}} + \boldsymbol{\tau}_A + \boldsymbol{\tau}_D. \quad (1.17)$$

$\boldsymbol{\tau}_A$ is the resulting torque induced by the aerodynamic forces distributed when they do not have an application point coinciding with the center of gravity of the vehicle, combined with the rotational drag that the air opposes to the rotations of the rigid body on roll, pitch and yaw. $\boldsymbol{\tau}_D$ is the moment produced by the disturbances.

Definition 1.1.1 : State of the Quadrotor

Given the previous premises, we define the state of a quadrotor as a vector in $\mathbb{R}^{12}$ defined as follows:

$$\boldsymbol{\xi} = (x, y, z, v_x, v_y, v_z, \varphi, \vartheta, \psi, p, q, r)^T \quad (1.18)$$

here x, y, z is the position of the center of mass of the quadrotor expressed in SR_i , v_x, v_y, v_z its velocity, φ, ϑ, ψ the roll-pitch-yaw angles between SR_i and SR_b , and p, q, r the angular velocity of the quadrotor expressed in SR_b .

The evolution in time of the quadrotor state is given by a dynamical system of the following form:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \quad (1.19)$$

where $\mathbf{u}$ are the inputs of the system defined as follows:

$$\mathbf{u} = \begin{pmatrix} T \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \quad (1.20)$$

these quantities depend on $f_i, \tau_{R,i}$, which in turn depend on the angular velocity of the rotors ω_i , which we assume we can control perfectly.

Definition 1.1.2 : Time Evolution of the Quadrotor

Let Ω_r be the average blades rotation velocity, and I_r the blades inertia, the explicit expression of the system in Equation (1.19) is the following:

$$\dot{x} = v_x \quad (1.21)$$

$$\dot{y} = v_y \quad (1.22)$$

$$\dot{z} = v_z \quad (1.23)$$

$$\dot{v}_x = F_{A,x} - (\cos(\psi) \sin(\vartheta) \cos(\varphi) + \sin(\psi) \sin(\varphi)) \frac{T}{m} \quad (1.24)$$

$$\dot{v}_y = F_{A,y} - (\sin(\psi) \sin(\vartheta) \cos(\varphi) - \sin(\varphi) \cos(\psi)) \frac{T}{m} \quad (1.25)$$

$$\dot{v}_z = F_{A,z} + g - \cos(\vartheta) \cos(\varphi) \frac{T}{m} \quad (1.26)$$

$$\dot{\varphi} = p + \sin(\varphi) \tan(\vartheta) q + \cos(\varphi) \tan(\vartheta) r \quad (1.27)$$

$$\dot{\vartheta} = \cos(\varphi) q - \sin(\varphi) r \quad (1.28)$$

$$\dot{\psi} = \sin(\varphi) \sec(\vartheta) q + \cos(\varphi) \sec(\vartheta) r \quad (1.29)$$

$$\dot{p} = \tau_{A,x} + \frac{I_r}{I_x} q \Omega_r + \frac{I_y - I_z}{I_x} q r + \frac{\tau_\varphi}{I_x} \quad (1.30)$$

$$\dot{q} = \tau_{A,y} + \frac{I_r}{I_y} p \Omega_r + \frac{I_z - I_x}{I_y} p r + \frac{\tau_\vartheta}{I_y} \quad (1.31)$$

$$\dot{r} = \tau_{A,z} + \frac{I_x - I_y}{I_z} p q + \frac{\tau_\psi}{I_z}. \quad (1.32)$$

1.2 Quadrotor Control

The model in Definition 1.1.2 is too complex for control purposes. We neglect aerodynamics and gyroscopic effects, and we assume small changes for φ and ϑ such that

$$\begin{pmatrix} \dot{\varphi} & \dot{\vartheta} & \dot{\psi} \end{pmatrix}^T \simeq \begin{pmatrix} p & q & r \end{pmatrix}^T \quad (1.33)$$

we assume that the shape is symmetric and we neglect the disturbances.

Definition 1.2.1 : Simplified Time Evolution of the Quadrotor

$$\ddot{x} = -(\cos(\psi) \sin(\vartheta) \cos(\varphi) + \sin(\psi) \sin(\varphi)) \frac{T}{m} \quad (1.34)$$

$$\ddot{y} = -(\sin(\psi) \sin(\vartheta) \cos(\varphi) - \sin(\varphi) \cos(\psi)) \frac{T}{m} \quad (1.35)$$

$$\ddot{z} = -\cos(\vartheta) \cos(\varphi) \frac{T}{m} + g \quad (1.36)$$

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} \quad (1.37)$$

$$\ddot{\vartheta} = \frac{\tau_\vartheta}{I_y} \quad (1.38)$$

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \quad (1.39)$$

Note how we changed the notation of the acceleration ($\ddot{x} = \dot{v}_x$, $\ddot{y} = \dot{v}_y$, $\ddot{z} = \dot{v}_z$). The basic control

objectives for a quadrotor are the following:

- attitude control,
- height control,
- position control,
- hovering-to-hovering motion,
- waypoint navigation,
- trajectory planning and tracking,
- obstacle avoidance.

With **attitude control**, we intend the process of determining the torques $\tau_\varphi, \tau_\vartheta, \tau_\psi$ needed to obtain a stable desired attitude $\varphi_d, \vartheta_d, \psi_d$. A trivial solution is a proportional-derivative (PD) action on the error.

Definition 1.2.2 : PD Controller for the Attitude

$$\tau_\varphi = K_{\varphi p}(\varphi_d - \varphi) + K_{\varphi d}(\dot{\varphi}_d - \dot{\varphi}) \quad (1.40)$$

$$\tau_\vartheta = K_{\vartheta p}(\vartheta_d - \vartheta) + K_{\vartheta d}(\dot{\vartheta}_d - \dot{\vartheta}) \quad (1.41)$$

$$\tau_\psi = K_{\psi p}(\psi_d - \psi) + K_{\psi d}(\dot{\psi}_d - \dot{\psi}). \quad (1.42)$$

For the **height control** we consider the dynamic model, in particular the Equation (1.36). We choose the value of T that linearizes the system.

Definition 1.2.3 : Height Control with FBL

To bring and keep the quadrotor to a desired height z_d we apply the feedback linearization plus a PD controller.

$$T = \frac{m}{\cos \vartheta \cos \varphi} [g - \ddot{z}_d - K_{zp}(z_d - z) - K_{zd}(\dot{z}_d - \dot{z})]. \quad (1.43)$$

Observation 1.2.1

A quadrotor is an underactuated system: it has 6 degrees of freedom in space (x, y, z and three attitude angles), but only 4 independent control inputs (the total thrust T and the three torque moments $\tau_\varphi, \tau_\vartheta, \tau_\psi$). There are no side motors to push the drone directly along the x axis or the y axis. To translate horizontally, the drone must first tilt (rotate) so that a thrust component T points in the desired direction.

Given the underactuated nature of the quadrotor, while we are controlling the altitude z we can't directly control x and y , hence we adopt a *hierarchical* control scheme.

1. First of all we assume that we have direct control on x and y , with two **virtual control inputs** u_x and u_y :

$$\ddot{x} = u_x \quad (1.44)$$

$$\ddot{y} = u_y. \quad (1.45)$$

2. These virtual control inputs are defined as a PD action on the error for stabilization:

$$u_x = \ddot{x}_d + K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) \quad (1.46)$$

$$u_y = \ddot{y}_d + K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}). \quad (1.47)$$

3. From the dynamics in Equations (1.34) and (1.35), we denote T_x and T_y the normalized components of the thrust along the inertial axes x and y :

$$\ddot{x} = \underbrace{(-\cos \varphi \sin \vartheta \cos \psi - \sin \varphi \sin \psi)}_{T_x} \frac{T}{m} = T_x \frac{T}{m} \quad (1.48)$$

$$\ddot{y} = \underbrace{(-\cos \varphi \sin \vartheta \sin \psi + \sin \varphi \cos \psi)}_{T_y} \frac{T}{m} = T_y \frac{T}{m}. \quad (1.49)$$

4. Let's equate the real expressions to the virtual accelerations:

$$T_x \frac{T}{m} = u_x \implies T_x = \frac{m}{T} u_x = \frac{m}{T} [K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) + \ddot{x}_d] \quad (1.50)$$

$$T_y \frac{T}{m} = u_y \implies T_y = \frac{m}{T} u_y = \frac{m}{T} [K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}) + \ddot{y}_d] \quad (1.51)$$

in this way we get the desired value for T_x and T_y in order to achieve u_x and u_y .

5. At this point we have to solve (1.48) and (1.49) for the roll and the pitch to get the desired values φ_d and ϑ_d :

$$\varphi_d = \arcsin(T_y \cos \psi - T_x \sin \psi) \quad (1.52)$$

$$\vartheta_d = -\arcsin\left(\frac{T_x \cos \psi + T_y \sin \psi}{\cos \varphi_d}\right). \quad (1.53)$$

6. In the end the inner loop will generate the desired values for $(\tau_\varphi, \tau_\vartheta, \tau_\psi)$ to achieve φ_d and ϑ_d .

To recap the whole control scheme:

- The quadrotor has no laterally oriented propellers, but generates a single main thrust T perpendicular to its frame. Consequently, there is no independent direct force on x and y .
- T is set as a priority to support the weight of the quadrotor and govern the z altitude.
- If we keep the drone perfectly horizontal, the thrust only points upwards. If, however, we tilt it to roll (φ) and pitch (ϑ), the thrust vector breaks down: one part continues to compensate for gravity along z , while the other is projected onto the horizontal plane.
- The position loop calculates how much horizontal acceleration $(\ddot{x}, \ddot{y})$ is needed to reach the target and, working backwards, determines exactly which roll and pitch angles (φ_d, ϑ_d) project the necessary amount of thrust T in that precise direction.

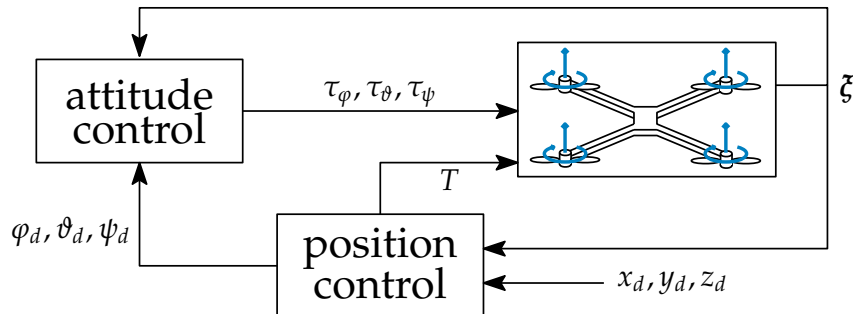


Figure 1.6: Structure of the control system.